

AdventQuest

Contexte complet pour Claude Code

Calendrier de l'Avent interactif — 24 mini-jeux quotidiens

React Native (Expo) · Phaser.js · Supabase

Auteure : Clervie

1. Concept & règles du jeu

Pitch

AdventQuest est une app mobile iOS/Android de calendrier de l'avent avec 24 mini-jeux quotidiens. L'histoire : le **Cœur de Noël** a été brisé en 24 fragments par l'esprit farceur **Grimnoir**. Le joueur (le Gardien des Fêtes) doit récupérer un fragment chaque jour en réussissant le mini-jeu du jour.

Règles fondamentales

- Chaque mini-jeu est **rejouable autant de fois que voulu le jour même**, puis verrouillé définitivement le lendemain.
- Si le joueur rate un jour ou ne joue pas, le fragment est **perdu définitivement**.
- Il faut **12 fragments minimum sur 24** pour débloquer le portail du Jour 24 (boss final).
- Pas de rejouer : une fois le jour suivant arrivé, c'est terminé.

Zones narratives

Zone	Jours	Lieu	Difficulté
■ Le Village Englouti	1–5	Village de Noël figé	■ Facile
■ La Forêt Gelée	6–10	Forêt enchantée	■ Moyen
■ Les Cavernes de Givre	11–15	Grottes de cristal	■ Moyen+
■ La Citadelle de Grimnoir	16–20	Forteresse de glace	■ Difficile
■ Le Pont Suspendu	21–23	Pont fragile	■ Très difficile
■ Salle du Cœur	24	Salle finale	■ Boss

Système de hints

- Jours 1–5 : 1 hint gagné par jour complété
- Jours 6–15 : 1 hint tous les 2 jours
- Jours 16–23 : aucun hint gagné, phase de dépense uniquement
- Jour 24 (boss) : aucun hint utilisable

Chaque jeu a sa propre utilisation du hint (révéler une case, éliminer des réponses, ralentir le jeu...).

2. Stack technique

Couche	Technologie	Rôle
App mobile	React Native (Expo)	UI, navigation, logique métier
Jeux arcade	Phaser.js (via WebView)	Runner, Casse-briques, etc.
State global	Zustand	Progression, hints, fragments
Backend	Supabase (self-hosted sur OVH)	Auth, DB, leaderboard
Notifications	Expo Notifications	Rappel quotidien
Animations	React Native Reanimated	Transitions, effets
Gestes	react-native-gesture-handler	Swipe (snake, labyrinthe...)
SVG	react-native-svg	Dessin connecté
Tests	Jest (jest-expo preset)	Logique pure de chaque jeu

Dépendances installées

```
npx expo install expo-router react-native-reanimated react-native-webview
npx expo install expo-notifications react-native-gesture-handler
npx expo install react-native-svg @react-native-async-storage/async-storage
npm install zustand @supabase/supabase-js
```

Variables d'environnement (.env)

```
EXPO_PUBLIC_SUPABASE_URL=https://xxx.supabase.co
EXPO_PUBLIC_SUPABASE_PUBLISHABLE_KEY=sb_publishable_...
```

■ ■ Utiliser la publishable key (ancienne anon key), jamais la secret key dans le code client.

3. Architecture du projet

Structure des dossiers

```
AdventQuest/
├── app/
│   ├── (tabs)/
│   │   ├── _layout.tsx      → 3 onglets (Calendrier, Classement, Profil)
│   │   ├── index.tsx       → écran Calendrier
│   │   ├── leaderboard.tsx  → écran Classement
│   │   ├── profile.tsx     → écran Profil
│   │   ├── game/[day].tsx   → écran jeu dynamique
│   │   └── _layout.tsx     → layout racine (Stack global)
│   ├── components/
│   │   ├── Calendar/       → grille 24 cases
│   │   ├── DayCell/        → case individuelle
│   │   ├── GameWrapper/    → wrapper commun tous jeux RN
│   │   │   ├── index.tsx
│   │   │   └── types.ts     → GameResult, GameComponentProps
│   │   ├── PhaserGameWrapper/ → wrapper WebView pour jeux Phaser
│   ├── games/
│   │   ├── day01_quiz/
│   │   ├── day02_stack/    → Phaser (game.html + index.tsx)
│   │   ├── day03_sudoku/
│   │   └── ... (19 dossiers)
│   ├── store/
│   │   └── gameState.ts     → Zustand : progression, hints, fragments
│   ├── services/
│   │   └── supabase.ts
│   ├── constants/
│   │   └── days.ts          → config des 24 jours
```

Interface commune — tous les jeux respectent ce contrat

```
// components/GameWrapper/types.ts
export interface GameResult {
  success: boolean;
  score: number;
}
export interface GameComponentProps {
  onGameEnd: (result: GameResult) => void;
  hintsAvailable: number;
  onUseHint: () => void;
  difficulty?: 'easy' | 'medium' | 'hard' | 'very_hard';
}
```

Registre des jeux — app/game/[day].tsx

```
const GAME_COMPONENTS: Record<string, GameComponentProps> = {
  quiz: QuizGame,
  sudoku: SudokuGame,
  memory_sequence: MemorySequenceGame,
  snake: SnakeGame,
  stack: StackGame,           // Phaser
  labyrinthe: LabyrintheGame,
```

```
pipepuzzle: PipePuzzleGame,  
spaceinvaders: SpaceInvadersGame, // Phaser  
whackamole: WhackAMoleGame, // Phaser  
dessinconnecte: DessinConnecteGame,  
fruitninja: FruitNinjaGame, // Phaser  
bubbleshooter: BubbleShooterGame, // Phaser  
solitaire: SolitaireGame,  
nonogram: NonogramGame,  
rythme: RythmeGame,  
// À ajouter : runner, casse_briques, dodge_ball  
};
```

Store Zustand — gameStore.ts

```
{  
  currentDay: number, // depuis date serveur  
  hints: number, // hints disponibles  
  days: Record<number, {  
    fragmentWon: boolean,  
    bestScore: number,  
    attempts: number  
  }>,  
  canPlay: (day) => boolean, // day === currentDay  
  isLocked: (day) => boolean, // day > currentDay  
  totalFragments: () => number,  
  bossUnlocked: () => boolean, // fragments >= 12  
  finishAttempt: (day, score, success) => void,  
  useHint: () => void,  
  setCurrentDay: (day) => void,  
}
```

4. Planning des 24 jours

Légende difficulté : ■ Facile · ■ Moyen · ■ Difficile · ■ Très difficile · ■ Boss

Moteur : RN = React Native pur · Ph = Phaser via WebView

Jour	Jeu	Moteur	Diff.	Fragment	Zone
1	Quiz Noël (5 questions)	RN	■	Éclat de lumière	Village
2	Stack (empiler plateformes)	Ph	■	Graine de givre	Village
3	Piano Tiles Noël	Ph	■	Cristal de nombre	Village
4	Space Invaders Noël	Ph	■	Souffle hivernal	Village
5	Sudoku 9×9 (facile)	RN	■	Racine ancienne	Village
6	Dessin connecté	RN	■	Clochette brisée	Forêt
7	Fruit Ninja Noël	Ph	■	Symbole de givre	Forêt
8	Bubble Shooter	Ph	■	Étoile de glace	Forêt
9	Runner de Noël	Ph	■	As de neige	Forêt
10	Pipe Puzzle	RN	■	Miroir brisé	Forêt
11	Solitaire Klondike	RN	■	Pierre de patience	Cavernes
12	Sudoku 9×9 (moyen)	RN	■	Cristal de mémoire	Cavernes
13	Snake Croque-Pommes (45s)	RN	■	Engrenage d'or	Cavernes
14	Labyrinthe (8×8)	RN	■	Sablier gelé	Cavernes
15	Nonogram 5×5	RN	■	Lien d'argent	Cavernes
16	Whack-a-Mole	Ph	■	Écaille de dragon	Citadelle
17	Sudoku 9×9 (difficile)	RN	■	Larme de fée	Citadelle
18	Memory Séquence (Simon)	RN	■	Flèche de lumière	Citadelle
19	Dodge Ball (esquiver 60s)	Ph	■	Braise éternelle	Citadelle
20	Casse-briques enchanté	Ph	■	Médaille de bronze	Citadelle
21	Jeu de rythme	RN	■	Dernier éclat	Pont
22	Marathon : Space Inv. + Bubble	Ph+Ph	■	(score combiné)	Pont
23	Marathon : Dodge Ball + Nonogram	Ph+RN	■	(dernier frag.)	Pont
24	Boss : Stack + Casse-briques + Quiz	Ph+Ph+RN	■	Le Cœur de Noël	Sommet

5. État actuel — jeux implémentés (15/19 mécaniques)

Jeux React Native — terminés et testés

Jeu	Dossier	Tests	Notes
Quiz Noël	games/day01_quiz/	■	5 questions QCM, hint élimine 2 mauvaises réponses
Sudoku 9×9	games/day03_sudoku/	■	Backtracking, difficulté = nb cases retirées (30/40/50/58)
Memory Séquence	games/day04_memory_sequence/	■	Simon-like, séquence s'allonge à chaque niveau
Snake Croque-Pommes	games/day05_snake/	■	45s, pas de mort, serpent traverse les murs
Dessin connecté	games/day06_dessinconnecte/	■	SVG + .runOnJS(true) requis pour swipe
Pipe Puzzle	games/day10_pipepuzzle/	■	Tuiles rotatives, BFS pour valider la connexion
Solitaire Klondike	games/day11_solitaire/	■	Tap-to-select/place (pas drag&drop), 30+ tests
Labyrinthe 8×8	games/day14_labyrinthe/	■	Recursive backtracking + test BFS solvabilité
Nonogram	games/day15_nonogram/	■	Cycles 3 états : empty→filled→marked→empty
Jeu de rythme	games/day21_rythme/	■	requestAnimationFrame, SANS audio pour l'instant

Jeux Phaser (WebView) — terminés et testés

Jeu	Dossier	Notes
Stack	games/day02_stack/	Plateformes oscillantes, découpage du débordement
Piano Tiles	games/day03_pianoTiles/	Tuiles noires qui tombent, 4 colonnes
Space Invaders Noël	games/day04_spaceinvaders/	3 vagues en easy, tir avec cooldown 350ms
Fruit Ninja Noël	games/day07_fruitninja/	Distance segment-cercle pour swipe rapide
Bubble Shooter	games/day08_bubbleshooter/	Grille hexa, flood fill double (match + flottants)
Whack-a-Mole	games/day16_whackamole/	Grille 3×3, faux fragments en diff. medium+

■■ IMPORTANT — Tous les jeux RN avec `Gesture.Pan()` doivent avoir `.runOnJS(true)` sur le gesture, sinon le swipe ne fonctionne pas (callback sur le thread UI natif).

Infrastructure commune — terminée

- GameWrapper (3 phases : intro / jeu / résultat, règle rejouabilité)
- PhaserGameWrapper (pont WebView ↔ RN via postMessage)
- Store Zustand (progression, hints, fragments, canPlay)
- Registre GAME_COMPONENTS dans app/game/[day].tsx
- Setup Jest avec tests de logique pure pour chaque jeu RN
- Template Phaser réutilisable (games/_phaser_template/template.html)
- Protocole de messages RN↔Phaser : GAME_READY / GAME_OVER / REQUEST_HINT_USE / INIT

6. Jeux restants à implémenter

Jeux Phaser — 3 restants

Jour 9 — Runner de Noël (Phaser)

- Personnage qui court automatiquement, vitesse augmente progressivement
- Tap/double-tap : saut. Swipe bas : glissade
- Génération procédurale d'obstacles (racines, stalactites)
- Timer 60 secondes, score = distance + fragments collectés
- Difficulté : vitesse de départ + taux d'obstacles

```
games/day09_runner/ → game.html + index.tsx
```

Jour 19 — Dodge Ball (Phaser)

- Personnage en bas, déplacement gauche/droite par swipe ou pointermove
- Projectiles de Grimnoir tombent depuis le haut, vitesse et fréquence croissantes
- Timer 60–90 secondes, score = temps survécu × multiplicateur
- Game over si touché par un projectile

```
games/day19_dodgeball/ → game.html + index.tsx
```

Jour 20 — Casse-briques enchanté (Phaser)

- Raquette en bas contrôlée par pointermove, balle rebondissante
- Blocs de glace en grille en haut, certains avec bonus/malus
- Bonus : balle double, raquette large, +10s
- Malus : inversion commandes, balle rapide
- Game over si la balle tombe en bas (1 vie)
- Score = blocs détruits × multiplicateur

```
games/day20_cassebriques/ → game.html + index.tsx
```

Jeux RN — 1 restant (à finaliser/améliorer)

Jeu de rythme — ajouter l'audio (expo-av)

- Installer expo-av : `npx expo install expo-av`
- Synchroniser `startTimeRef` avec le lancement réel du fichier audio
- Créer 2 partitions .mp3 de mélodies de Noël (carillon, valse)
- La logique de jeu (`charts.ts`, `logic.ts`) n'a PAS besoin de changer

7. Phases restantes du projet

Phase 4 — Contenu (à faire après tous les jeux)

- Rédiger les 24 textes d'intro (storyIntro de chaque jour dans constants/days.ts)
- Rédiger les 6 textes de zone (bannière narrative par zone)
- Écrire 50+ questions pour le Quiz Noël (avec vérification des réponses)
- Créer les illustrations par zone (6 backgrounds) via MidJourney ou DALL-E
- Assets des jeux Phaser : sprites lutins, ornements, créatures de Grimnoir
- Sons d'interface : ouverture de case, victoire, échec, fragment obtenu
- Mélodies de Noël pour le jeu de rythme (.mp3)
- Compléter constants/days.ts pour les 24 jours (game, difficulty, fragment, story)

Phase 5 — Backend Supabase

Schéma de base de données

```
users (id, email, username, avatar_url)
player_progress (user_id, day, completed, fragment_won, best_score, attempts)
leaderboard (user_id, total_score, fragments_count, streak)
player_hints (user_id, hints_available, hints_used)
friendships (user_id, friend_id, status)
```

Points critiques

- currentDay doit venir de la date SERVEUR (pas du téléphone — anti-triche)
- Edge Function validate-score : vérifier que day === today côté serveur
- Row Level Security (RLS) sur toutes les tables
- Subscription temps réel pour le leaderboard
- Remplacer les données mockées dans gameStore.ts par les vraies requêtes Supabase

Phase 6 — Polish & tests

- Animations d'ouverture de case (rotation + particules confettis)
- Animation fragment obtenu (cristal qui se place dans le Cœur)
- Neige tombante animée en arrière-plan du calendrier
- Barre de progression fragments avec marqueur seuil 12
- Drag & drop amélioré pour le Solitaire (optionnel, actuellement tap-to-move)
- Tests visuels sur différentes tailles d'écran (iPhone SE, iPhone 15, Android petit/grand)
- Performance check : vérifier que les jeux Phaser ne lagent pas sur vieux Android

Phase 7 — Publication

- Compte Apple Developer (99\$/an) — commencer tôt, délai de validation possible

- Compte Google Play Console (25\$ unique)
- EAS Build : `eas build --platform ios --profile production`
- EAS Build : `eas build --platform android --profile production`
- Screenshots iOS (6.7", 6.1", iPad) et Android
- Description app FR + EN, métadonnées stores
- Review Apple (24–48h minimum, parfois 7 jours)

8. Conventions de développement

Workflow par jeu (à respecter pour chaque nouveau jeu)

Étape	Action	Fichier
1	Écrire la logique pure	games/dayXX_nom/logic.ts
2	Écrire les tests	games/dayXX_nom/logic.test.ts
3	Valider avec npm run test:ci	—
4	Construire le composant visuel	games/dayXX_nom/index.tsx
5	Ajouter au registre	app/game/[day].tsx → GAME_COMPONENTS
6	Configurer le jour	constants/days.ts

Pièges connus et solutions

■ Swipe qui ne fonctionne pas (jeux RN)

Tous les `Gesture.Pan()` doivent avoir `.runOnJS(true)` — le callback s'exécute sur le thread UI, pas le thread JS, ce qui bloque les appels `setState`.

```
const gesture = Gesture.Pan()
  .runOnJS(true)
  .onEnd((event) => { ... })
```

■ Variables d'environnement non reconnues

Redémarrer expo avec `--clear` après toute modification du `.env`. Les variables doivent commencer par `EXPO_PUBLIC_`.

```
npx expo start --clear
```

■ Stale closures avec `setInterval` (Snake, Jeu de rythme)

Utiliser `useRef` pour les valeurs lues dans un `setInterval/requestAnimationFrame`, jamais les valeurs `useState` directement.

```
const directionRef = useRef(direction);
directionRef.current = direction; // met à jour à chaque render
```

■ Variables d'environnement non reconnues (Expo)

`metro.config.js` doit déclarer les fichiers `.html` comme assets pour charger les jeux Phaser :

```
config.resolver.assetExts.push('html');
```

■ Barre de statut qui cache le contenu

Utiliser `useSafeAreaInsets()` sur chaque écran principal avec contenu en haut de page.

```
const insets = useSafeAreaInsets();
```

Convention de communication Phaser ↔ React Native

Direction	Message	Déclencheur
Phaser → RN	{ type: 'GAME_READY' }	Jeu chargé, cache le spinner
Phaser → RN	{ type: 'GAME_OVER', success, score }	Fin de partie
Phaser → RN	{ type: 'REQUEST_HINT_USE' }	Joueur utilise un hint
RN → Phaser	{ type: 'INIT', difficulty }	Au chargement, paramètre la difficulté

9. Checklist de validation globale

Avant de passer en Phase 4 (contenu)

- ■ Runner de Noël implémenté et testé (iOS + Android)
- ■ Dodge Ball implémenté et testé (iOS + Android)
- ■ Casse-briques implémenté et testé (iOS + Android)
- ■ Solitaire validé visuellement (tap-to-move fonctionnel, toutes les règles Klondike respectées)
- ■ Jeu de rythme validé avec audio (expo-av intégré)
- ■ constants/days.ts complété pour les 24 jours
- ■ npm run test:ci passe entièrement (tous les jeux RN)

Avant la publication (Phase 7)

- ■ currentDay câblé sur la date serveur (Supabase Edge Function)
- ■ Auth complète (inscription, connexion, persistance de session)
- ■ Leaderboard connecté à Supabase (temps réel)
- ■ Notifications push configurées
- ■ Testé sur iPhone SE (petit écran), iPhone 15, Android petit et grand
- ■ Aucun crash en mode hors ligne
- ■ Compte Apple Developer créé (délai possible)
- ■ Build EAS iOS et Android sans erreur

Rappel — MVP pour Noël 2025 (si temps limité)

Si le temps manque, 12 jours suffisent pour un MVP :

Jour	Jeu	Priorité
1	Quiz Noël	Indispensable — introduction
2	Stack	Signature du jeu
4	Space Invaders	Action dès le début
5	Sudoku 9×9	Classique attendu
7	Fruit Ninja	Fun immédiat
9	Runner	Showpiece Phaser
11	Solitaire	Jeu de cartes
13	Snake	Nostalgie
16	Whack-a-Mole	Zone 4
19	Dodge Ball	Tension
20	Casse-briques	Final zone 4

24	Boss (2 phases)	Conclusion
----	-----------------	------------

Les 12 autres jours affichent "À venir — Saison 2" avec un design mystérieux.